

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Oberst Eduard, FAF-243

Report

Laboratory work nr. 6

Inside the xv6 File System

Course: Operating Systems (Track B: xv6)

Checked by:
Patricia Reitman, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Purpose of the work

Lab 5 looked at a file system from the outside, with the commands of a running Linux. The purpose of this laboratory work is to look at the same ideas from the inside, in the few hundred readable lines of xv6: to read what `ls` shows and decode its columns against `kernel/stat.h`, to write `mystat`, a program of my own that calls the `stat()` system call and prints the inode, the link count and the size, to watch hard links behave in the kernel's own numbers, to measure the limits that are written as constants in `kernel/fs.h` and printed by `mkfs`, and finally to write `find`, a program that walks the directory tree recursively.

All work was done in `~/xv6-riscv` on Ubuntu 26.04.1 LTS, with the `riscv64-linux-gnu` cross toolchain and QEMU emulating the RISC-V machine. Every new program is one file `user/NAME.c` plus one line `$U/_NAME\` in the `UPROGS` list of the Makefile, with a TAB at the start and nothing after the backslash; `make` then compiles it and `mkfs` copies it into the disk image `fs.img`. QEMU is left with Ctrl-A followed by X. In the figures the xv6 shell prompt is the bare `$`, and the host prompt is shortened to `~/xv6-riscv$`.

Preparation

```
~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
-nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,
format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0


xv6 kernel is booting


hart 1 starting
hart 2 starting
init: starting sh
$
```

The machine has three harts and 128 MB of memory, and the disk is the single file `fs.img` attached as a virtio block device. One detail matters for the whole lab: `fs.img` survives between runs, so a test that depends on a clean disk has to be preceded by `make clean`, which deletes the image and makes `mkfs` build a fresh one.

Part 1. Read what xv6 shows you

Interesting Command Output:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
sleep          2 23 35280
args           2 24 35272
console        3 25 0
f              2 26 6
d              1 27 32
```

Figure 1.1 – The root directory: name, type, inode, size

```
$ mkdir a
$ ls a
.              1 28 32
..             1 1 1024
```

Figure 1.2 – A new, empty directory

The last two lines of Figure 1.1, `f` and `d`, are not part of xv6: they are leftovers of an earlier session that stayed in `fs.img` because the image is only rebuilt by `make clean`. They are useful anyway, because `d` is an empty directory of size 32 made outside this experiment and it shows the same size as the `a` of Figure 1.2.

Observations:

1. **What do types 1, 2 and 3 mean?**

   They are the three constants of `kernel/stat.h`: `T_DIR 1`, `T_FILE 2` and `T_DEVICE 3` and they are exactly the `type` field of `struct stat`. In the listing every program

is type 2, `.`, `..` and `d` are type 1, and `console` is the only type 3. The device line is the interesting one: its size is 0 because a device has no data blocks at all – reads and writes are routed to a driver through the `devsw` table, not to the disk. xv6 has no equivalent of the Linux `l` (symbolic link), `p`, `s` or `b`: only these three types exist.

2. **Which inode number does the root directory have, and why are . and .. the same there?**

   The root is inode 1, which `kernel/fs.h` fixes as `#define ROOTINO 1` and `mkfs` asserts when it allocates the first inode. Both `.` and `..` in the root are 1 because the root is its own parent: `mkfs` writes the two entries pointing at `rootino` itself. This is what terminates path resolution at the top, so `cd ..` in `/` stays in `/` and there is no way to walk out of the tree. Figure 1.2 shows the normal case for comparison: inside `a`, `.` is 28, the inode of `a`, while `..` is 1, the root.

3. **A directory entry has a fixed size. How big is one entry, and which two entries does a contain?**

   A directory is an ordinary file whose content is a sequence of `struct dirent`, and that structure is `ushort inum` plus `char name[DIRSIZ]` with `DIRSIZ 14`, so one entry is $2 + 14 = 16$ bytes. A new directory contains exactly two entries, `.` pointing at itself and `..` pointing at its parent, hence $2 \times 16 = 32$ bytes, which is the size `ls` reports for both `a` and `d`. Each name is a fixed 14-byte field rather than a string, which is why it is padded with zero bytes and declared `__attribute__((nonstring))`: it need not end in NUL. The root is 1024 for a different reason – it is not full of entries, but `mkfs` deliberately rounds its size up to a whole block at the end of the build (`mkfs/mkfs.c:169-173`).

Table 1.1 – The three file types of xv6

Type	Constant	What it is, and how it appears in Figure 1.1
1	`T_DIR`	Directory – a file whose content is `struct dirent` entries.
2	`T_FILE`	Regular file – `README` and all the user programs.
3	`T_DEVICE`	Device – only `console`; size 0, served by a driver.

Part 2. Write mystat

`stat()` fills a `struct stat` with `dev`, `ino`, `type`, `nlink` and `size`. The program loops over its arguments, reports a failure on file descriptor 2 and continues, and translates the

type number with a small helper. One detail of xv6 matters here: `size` is a `uint64`, while the xv6 `printf` understands only `%d` on an `int`, so the field has to be cast – without the cast the remaining arguments of the same call are printed wrong.

Interesting Command Output:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"


char *
typename(int t)
{
  switch (t) {
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  }
  return "?";
}


int
main(int argc, char *argv[])
{
  struct stat st;


  if (argc < 2) {
    fprintf(2, "usage: mystat file...\n");
    exit(1);
  }
  for (int i = 1; i < argc; i++) {
    if (stat(argv[i], &st) < 0) {
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), (int)st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Figure 2.1 – Contents of user/mystat.c

```
~/xv6-riscv$ grep -n "_mystat\|_find" Makefile
155:    $U/_mystat\
156:    $U/_find\
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
```

Figure 2.2 – mystat on a file, a directory and a device

```
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Figure 2.3 – Hard links before and after rm

`console` is inode 25 in Figure 1.1 but inode 27 here. Nothing moved: `mkfs` allocates inodes in the order in which it copies files into the image, that order is the order of `UPROGS`, and adding `_mystat` and `_find` to the list pushed everything after them two places down. The inode number is a position inside this file system, not a property of the program.

Observations:

1. **The outputs after ln and after rm: what changed and what did not?**

   After `ln README r2` both names report `inode 2, links 2, size 2441`: one single file with two names. After `rm README` the remaining name reports `inode 2, links 1, size 2441`, and `cat r2 | wc` still prints `48 336 2441`, the same 2441 bytes. What changed is only the link count, and the fact that one entry is gone from the root directory. What did not change is everything that belongs to the file itself: the inode number, the size and the content. This is the Lab 5 experiment repeated with the kernel's own counters visible, and it makes the point sharper: `README` was never "the original" – the two names were of equal rank and `rm` simply removed one of them.

2. **Which line of sys_unlink decrements the link count, and when are the data blocks freed?**

   `kernel/sysfile.c:244`, the line `ip->nlink--;` followed by `iupdate(ip)`, right after the directory entry has been zeroed with `writei`. Four lines above it, at `:239`, there is a second decrement, `dp->nlink--`, which applies only when the victim is a directory and accounts for the `..` entry that disappears with it. Neither line frees a single block. The blocks are freed in `iput()` (`kernel/fs.c:350`), which calls `itrunc(ip)` at line 365 only when `ip->ref == 1 && ip->valid && ip->nlink == 0` – that is, when the last name AND the last open reference are both gone. This is exactly why a file that is deleted while a

process still has it open keeps working until that process closes it, and why the space comes back only then.

3. **xv6 has no symbolic links and no permissions. Which fields does an xv6 inode have?**

   `struct dinode` in `kernel/fs.h` has six fields: `type` (the three constants of Table 1.1), `major` and `minor` (meaningful only for `T_DEVICE`), `nlink`, `size`, and `addrs[NDIRECT + 1]`, the twelve direct block numbers plus the one indirect block measured in Part 3. What is absent is as informative as what is present: there is no owner, no group and no permission bits, so the whole of Part 3 of Lab 5 has no counterpart here; there are no timestamps, so nothing like the Access/Modify/Change/Birth lines of Linux `stat`; and there is no `T_SYMLINK`, so a name can only ever be a hard link. Fewer fields are also why `IPB = BSIZE / sizeof(struct dinode)` comes out at 16 inodes per block.

Part 3. Measure the limits

Interesting Command Output:

```
~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

Figure 3.1 – The constants of fs.h and the layout printed by mkfs

```
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
...
args           2 24 35272
mystat         2 25 36376
find           2 26 42632
console        3 27 0
averyveryveryl 1 28 32
```

Figure 3.2 – A name longer than DIRSIZ

Observations:

1. **What does ls show for averyveryverylongname, and which constant explains it?**

   It shows `averyveryveryl`: the 21-character name was silently cut to its first 14 characters, the value of `DIRSIZ`. The reason is the fixed `char name[DIRSIZ]` field of `struct dirent`: `dirlink` copies the name into it with `strncpy`, which writes at most 14 characters, and `namex` compares only that many. Nothing reports an error, which is the dangerous part – two names that differ only after the fourteenth character would be the same name to xv6, and the second `mkdir` would fail with a confusing "already exists". The entry is otherwise completely normal: type 1, inode 28, size 32.

2. **Compute the largest file xv6 can hold.**

   An inode holds `NDIRECT = 12` direct block numbers plus one indirect block. The indirect block is one block of `BSIZE = 1024` bytes filled with block numbers of `sizeof(uint) = 4` bytes, so it holds `NINDIRECT = 1024 / 4 = 256` of them. The maximum file is therefore `MAXFILE = NDIRECT + NINDIRECT = 12 + 256 = 268` blocks, that is $268 \times 1024 = 274\,432$ bytes, about 268 KB. `fs.h` states the same formula directly as `#define MAXFILE (NDIRECT + NINDIRECT)`. The number is worth comparing with the disk: the whole file system is 2000 blocks, so about seven files of maximum size would fill it, and `usertests`, the biggest program in the image at 206 800 bytes, is already at three quarters of the limit.

3. **From the nmeta line: how many blocks are there in total, and how are they divided?**

   `mkfs` prints `nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000`. The division is given in Table 3.1. The total of 2000 blocks of 1024 bytes is a disk of 2 048 000 bytes, which is exactly the size of `fs.img`, and the 47 metadata blocks are 2.4 % of it. The individual numbers all follow from `param.h` and `mkfs.c`: the log is `LOGBLOCKS + 1 = MAXOPBLOCKS * 3 + 1 = 31` blocks, one header plus thirty data blocks, so that any single file system operation (which writes at most 10 blocks) can be committed atomically; the inode area is `NINODES / IPB + 1 = 200 / 16 + 1 = 13` blocks for 200 inodes at 16 per block; and one bitmap block of 8192 bits is more than enough to track 1953 data blocks.

Table 3.1 – The layout of the 2000-block disk

Area	Blocks	Share	Purpose
Boot + super	2	0.1 %	Boot block (unused) and the superblock of Figure 3.1.
Log	31	1.6 %	The journal: header + 30 data blocks for atomic commits.
Inodes	13	0.7 %	200 inodes, 16 per block (`IPB = BSIZE / sizeof(dinode)`).
Bitmap	1	0.1 %	One bit per data block; 8192 bits in a block.
Data	1953	97.7 %	File and directory content.

Part 4. Write find

`find <dir> <name>` prints every path under `<dir>` whose last component is `<name>`. The program starts from `user/ls.c`, which already shows how to open a directory, read `struct dirent` entries one at a time and build a full path in a buffer, and adds the recursion.

Interesting Command Output:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"


void
find(char *path, char *name, int shownum)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;


  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if (fstat(fd, &st) < 0) {
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }
  if (st.type != T_DIR) {
    fprintf(2, "find: %s is not a directory\n", path);
    close(fd);
    return;
  }
  if (strlen(path) + 1 + DIRSIZ + 1 > sizeof buf) {
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }


  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';


  while (read(fd, &de, sizeof(de)) == sizeof(de)) {
    if (de.inum == 0)
      continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if (strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
      continue;
    if (stat(buf, &st) < 0) {
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if (strcmp(p, name) == 0) {
      if (shownum)
        printf("%s %d\n", buf, (int)st.ino);
      else
        printf("%s\n", buf);
    }
```

```
    if (st.type == T_DIR)
      find(buf, name, shownum);
  }
  close(fd);
}


int
main(int argc, char *argv[])
{
  int shownum = 0;


  if (argc < 3) {
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  if (argc > 3 && strcmp(argv[3], "-n") == 0)
    shownum = 1;
  find(argv[1], argv[2], shownum);
  exit(0);
}
```

Figure 4.1 – Contents of user/find.c

```
~/xv6-riscv$ make clean ; make qemu
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

Figure 4.2 – find on a fresh disk

Four points of the implementation are worth naming, because each of them is a trap:

- **Copy the name before comparing it.** `de.name` is a fixed 14-byte field declared `__attribute__((nonstring))`, so a name of exactly `DIRSIZ` characters has no terminating NUL and `strcmp` on it would read past the array. The entry is therefore copied into `buf` first and `p[DIRSIZ] = 0` terminates the copy, which is what `ls.c` does as well.
- **`p` is already the last component.** Because `buf` is built as `path + / +` name, the pointer `p` points at the final component of the path, so matching the target needs no helper that splits the path again.
- **Skip `.` and `..` before recursing.** Without this test the walk would descend into `.` for ever and climb out through `..` into the rest of the tree. Entries

with `inum == 0` are skipped too: those are free slots left behind by `sys_unlink`, which zeroes an entry instead of compacting the directory.

- **Close every descriptor on every path out.** `NOFILE` in `kernel/param.h` is 16, so a leaked descriptor per level would make the program fail around sixteen directories deep. The recursion itself has a second, lower limit: the xv6 user stack is a single 4096-byte page and each frame holds `buf[512]`, so the tree can be walked about six levels deep. Both limits are far above what this lab needs, but they are real.

The first attempt at the test failed with `link cat r2: failed`, because it was run on the disk left over from Part 2, where `r2` was still the second name of `README`. xv6's `link()` refuses to overwrite an existing name, so the test had to be repeated after `make clean` – another reminder that `fs.img` is persistent, and in passing a confirmation of Part 2: the name `r2` had outlived the `rm` of its twin.

Observations:

1. **Does find behave as the lab requires?**

   Yes. `find . r2` prints all three paths whose last component is `r2` – `./a/b/r2`, `./a/r2` and `./r2` – in the order in which the entries appear in the directories, and `find a b` prints `a/b`, showing that a directory is matched by name exactly like a file and that the search is not restricted to regular files. No `.` or `..` path is ever printed, and the program returns to the shell instead of looping.

Stretch: the -n option

```
$ find . r2 -n
./a/b/r2 31
./a/r2 32
./r2 3
$ ls
...
cat            2 3 36728
...
```

Figure 4.3 – find with inode numbers

2. **Which of the three r2 files are the same file as cat?**

   Only `./r2`. Its inode is 3, and Figure 1.1 shows `cat` at inode 3 as well: `ln cat r2` created a second name for the one inode, so `cat` and `./r2` are the same 36 728-byte file under two names, in the same way that `README` and `r2` were in Part 2. `./a/b/r2` (inode 31) and `./a/r2` (inode 32) were made by `echo` and are two separate two-byte files that merely share a name. This is the whole point of

printing the inode: the name says nothing about identity, and only the inode number distinguishes "the same file" from "another file with the same name".

Conclusion

This laboratory work showed the same file system ideas as Lab 5, but from the other side of the system call. In Lab 5 the inode was something `stat` reported; here it is `struct dinode` in `kernel/fs.h`, with its six fields, and the directory is visibly nothing but a file full of 16-byte `struct dirent` records. Writing `mystat` made the link count concrete: the same inode 2 appeared under two names after `ln`, and after `rm` of one of them the inode, the size and the content were untouched while only the count fell to 1. Reading `sys_unlink` explained why – the call removes a directory entry and decrements `nlink` at `sysfile.c:244`, and the data blocks are released somewhere else entirely, in `iput()`, and only when the last link and the last open reference are both gone.

Measuring the limits turned the constants into numbers with consequences. `DIRSIZ 14` is why a 21-character directory name came back as `averyveryveryl` with no error at all; twelve direct blocks plus 256 pointers in one indirect block cap a file at 268 KB; and the `nmeta` line shows that of 2000 blocks only 47 are metadata, of which the 31-block log is the largest single piece – the price xv6 pays so that every file system operation is committed atomically. The inode numbers themselves turned out to be positions rather than identities: adding two programs to `UPROGS` moved `console` from inode 25 to inode 27.

Writing `find` tied the two halves together. The recursion is short, but it only works because of what the earlier parts established: entries with `inum == 0` are free slots left by `unlink`, `.` and `..` must be skipped or the walk never ends, a 14-character name is not a C string, and descriptors have to be closed because a process may hold only 16. The `-n` option finally made the central idea of both labs visible in one line of output: three files called `r2`, and only the one at inode 3 is the same file as `cat`.